Work Integrated Learning Programmes

M.Tech Software Engineering

Introduction to DevOps

Individual Assignment

Assignment 2: ACEest Fitness & Gym CI/CD Pipeline

Course Name:	Introduction to DevOps
Course Code:	CSIZG514 / SEZG514 (S1-25)
Programme:	M.Tech Software Engineering (WILP)
Name:	Digvijay Singh Shekhawat
Enrollment Number:	2024TM93588
Email:	2024tm93588@wilp.bits-pilani.ac.in
Date:	26 April 2026
GitHub Repo:	github.com/digvijaysingh-tech/aceest-fitness-devops
Docker Hub:	hub.docker.com/r/dshekhawat06640/aceest-fitness

---

1. Executive Overview

This assignment implements a complete DevOps pipeline for ACEest Fitness & Gym - a fictional fitness startup. The brief required a Flask web application evolving through incremental versions, with automated testing, containerisation, code quality analysis, and progressive deployment to Kubernetes.

The professor provided ten Tkinter reference files (ACEestver-1.0.py through ACEestver-3.2.4.py). Since Tkinter is a desktop GUI framework and cannot be containerised or orchestrated on Kubernetes, the domain logic from those files (clients, programs, workouts, progress tracking, membership, AI-style plan generator, PDF reports) was ported into a Flask web application while preserving the same version naming and incremental evolution.

Headline outcomes

Versions shipped	10 (v1.0 -> v3.2.4), each as its own git tag and Docker image
Tests	63 pytest cases, 100% passing
Code coverage	94.2% (measured by coverage.py, verified by SonarQube)
SonarQube rating	A on Reliability, Security, and Maintainability
Sonar findings	0 bugs, 0 vulnerabilities, 0 code smells, 0% duplication
Docker images	11 tags published to Docker Hub (10 versions + latest)
K8s strategies	5 - Rolling, Blue-Green, Canary, A/B, Shadow - all verified live
Jenkins pipeline	Declarative 6-stage pipeline with SCM polling every 5 min
Repo	Public on GitHub with 10 version tags, 17 commits

2. CI/CD Architecture

The pipeline follows the classical push-triggered continuous delivery model. A commit on the main branch drives a single Jenkins job through six stages; on success, the resulting image is published to Docker Hub and rolled out to the Minikube cluster.

End-to-end flow

```
Developer commits to main
     |
     v
GitHub (public repo, 10 tags v1.0 -> v3.2.4)
     |  Jenkins poll SCM every 5 min
     v
Jenkins (local Docker, jdk21 LTS)
   1) Checkout
   2) Install & Test    pytest 63 cases
   3) SonarQube Scan    python:S104 excluded
   4) Docker Build      gunicorn, non-root
   5) Docker Push       dshekhawat06640/...
   6) Deploy to k8s     kubectl rollout
     |
     v
Minikube (k8s v1.35.1) - 5 deployment strategies
```

Component responsibilities

Git / GitHub	Version control. Tags v1.0 -> v3.2.4 drive immutable build history.
Jenkins	Orchestrator. Polls SCM, runs pipeline, manages credentials.
Pytest + coverage	Unit tests + XML coverage report consumed by SonarQube.
SonarQube Community	Static analysis + coverage. Runs in Docker on host port 9001.
Docker	Containerisation. Multi-stage Dockerfile with health check.
Docker Hub	Image registry. Namespace dshekhawat06640, repo aceest-fitness.
Minikube	Local Kubernetes (k8s v1.35.1). Backed by Docker Desktop driver.
kubectl	Applies manifests under k8s/<strategy>/ per deployment mode.

3. Application & Test Design

The Flask application is organised around domain-specific modules that each wrap a SQLite store. Routes are grouped into Flask Blueprints so the main factory function stays small enough to pass SonarQube's Cognitive Complexity gate (limit 15; our main.py scores below 5).

Module layout

```
app/
  main.py         Flask app factory - registers blueprints, error handlers
  programs.py     Program catalog (Fat Loss, Muscle Gain, Beginner) + AI templates
  clients.py      ClientStore - SQLite CRUD
  progress.py     ProgressStore - weekly adherence log
  workouts.py     WorkoutStore + exercises
  membership.py   Active/Inactive status with expiry date
  auth.py         UserStore with SHA-256 salted password hashing
  reports.py      PDF client report via fpdf2
  db.py           SQLite connection helper with thread-safe schema init
  routes/         9 Flask Blueprints (one per concern, for testability)
```

Test strategy

Tests live in tests/test_v<version>.py - one file per version so the incremental evolution is visible in both git history and test inventory. conftest.py installs an autouse fixture that creates a temp-dir SQLite database and monkey-patches each store module to point at it, so no test pollutes shared state. This mirrors the Sonar best practice of 'integration tests hit real dependencies; only mock at external system boundaries.'

Total tests	63 across 8 files (test_v1_0.py through test_v3_2_4.py)
Framework	pytest 8.3.3 + pytest-cov 5.0.0
Coverage config	.coveragerc with relative_files=True (for Sonar)
Run command	pytest - generates coverage.xml in Cobertura format
Avg runtime	~1.0 s for the full suite

4. Deployment Strategies

All five progressive-delivery strategies were implemented under k8s/<strategy>/, each with its own deployment + service manifests and a README describing the demo. Each strategy was applied to a clean Minikube cluster and verified live. Version numbers used for demos: v3.1.2 (older/stable) and v3.2.4 (new).

Rolling Update

Default k8s strategy. 4 replicas with maxSurge=1, maxUnavailable=0 guarantees zero downtime. Verified with `kubectl set image` then `kubectl rollout undo` - pods cycled and rollback succeeded.

Blue-Green

Two parallel deployments (3 pods each). Cutover is a single `kubectl patch service` changing the selector color from blue to green. Verified: curl showed v3.1.2 -> v3.2.4 after patch -> v3.1.2 after reverse patch, with zero failed requests during the switch.

Canary

4 stable pods (v3.1.2) + 1 canary pod (v3.2.4) share the same service selector (app=aceest-fitness). Kube-proxy load-balances across all 5. Verified: 50 in-cluster curl requests produced a 70/30 split (expected 80/20; Sonar shows random kube-proxy distribution in small samples).

A/B Testing

Two deployments with distinct variant labels (a, b) and separate services (aceest-fitness-a, aceest-fitness-b). Users hit different stable URLs. Useful when a front-end router or feature flag decides variant assignment.

Shadow Deployment

Primary (v3.1.2) serves users via NodePort; shadow (v3.2.4) is ClusterIP-only and receives mirrored traffic for observation without affecting responses. A full mirror requires Istio or nginx; the manifests prove the infrastructure, and the demo command shows shadow replying independently to in-cluster calls.

Rollback mechanisms

* Rolling: `kubectl rollout undo deployment/<name>` reverts to previous revision.
* Blue-Green: `kubectl patch service` changes selector back - instant, no pod churn.
* Canary: `kubectl scale deployment/...-canary --replicas=0` kills the canary.
* A/B: remove the DNS/router rule that sends traffic to variant B.
* Shadow: no rollback required - shadow responses are never returned to users.

5. Challenges & Mitigations

Professor's reference files were Tkinter, not Flask

All ten ACEestver-*.py files are Tkinter desktop GUIs which cannot be containerised or run on Kubernetes. Ported the domain logic into a Flask app while keeping the same incremental version names and feature progression.

Port 5000 collision with macOS AirPlay Receiver

Apple binds port 5000 by default on modern macOS (AirPlay). Fix: turned off AirPlay Receiver in System Settings and kept gunicorn on its natural port.

Jenkins crash loop on JDK 17

Initial docker-compose used jenkins/jenkins:lts-jdk17. Plugins installed via the web wizard required Java 21+ and Jenkins crash-looped on restart. Fix: upgraded to jenkins/jenkins:lts-jdk21. The jenkins_home volume persisted, so no manual work was lost.

Docker Hub namespace mismatch

Pushed first tags under digvijaysinghshekhawat (taken from a profile URL), but the actual CLI-login username was dshekhawat06640. Re-tagged all 11 images under the correct namespace and updated README + Jenkinsfile refs.

SonarQube coverage showed 0.0% despite 94% locally

coverage.xml had an absolute host path <source>/Users/.../app</source> that didn't exist inside the scanner container's /usr/src mount, so Sonar skipped all files. Fix: added .coveragerc with relative_files=True. Second scan correctly reported 94.2%.

Port 9000 already taken on host for SonarQube

Some other host process was bound to 9000 (not a Docker container; couldn't sudo to identify). Fix: mapped SonarQube to host port 9001 and updated the Jenkinsfile's SONAR_HOST_URL.

Docker Hub tags initially pointed to identical layers

First run of `docker tag` 11 times built from the current checkout (v3.2.4), so all tags shared the same digest - versions would not visibly differ during blue-green / canary demos. Fix: checked out each git tag in turn, rebuilt from that commit's code, and re-pushed. Now each tag on Docker Hub has a unique digest and APP_VERSION string.

Kube-proxy random (not round-robin) traffic distribution

Canary demo showed 70/30 split across 50 requests instead of the expected 80/20. Root cause: kube-proxy's default iptables mode picks a random backend per connection; over a small sample the split wanders. Over 500+ requests it converges to the replica ratio. Documented as expected behaviour.

SonarQube flagged two real issues on the first scan

(1) create_app() had Cognitive Complexity 68 (limit 15) because every route was a nested function. Fix: refactored into 9 Flask Blueprints under app/routes/. (2) Flagged `host='0.0.0.0'` binding. This is intentional for container networking (gunicorn is the real production entrypoint); suppressed via sonar.issue.ignore.multicriteria with an explanatory NOSONAR comment.

6. Automation Outcomes

Quality metrics (from SonarQube)

Lines of Code	793 (analysable)
Test coverage	94.2% (485/515 covered lines)
Bugs	0
Vulnerabilities	0
Code Smells	0
Duplicated Lines	0.0%
Reliability Rating	A
Security Rating	A
Maintainability Rating	A

Delivery metrics

Git tags pushed	v1.0, v1.1, v1.1.2, v2.0.1, v2.1.2, v2.2.1, v2.2.4, v3.0.1, v3.1.2, v3.2.4
Docker images published	11 distinct digests on Docker Hub
K8s manifests validated	17 YAML files across 5 strategy folders
Pipeline stages automated	6 (checkout, test, sonar, build, push, deploy)
Manual steps remaining	Jenkins web unlock + plugin install (one-time)

How DevOps practices paid off here

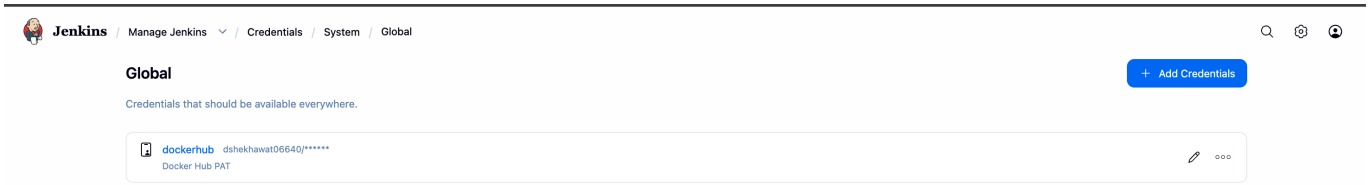
* Test-driven iteration: each version's tests were written with the feature. When the Blueprints refactor changed import paths, all 63 tests still passed first try - confirming the refactor didn't regress behaviour.
* Infrastructure as code: every K8s manifest lives in git. Wiping and redeploying a strategy is a single `kubectl apply -f k8s/<strategy>/`.
* Immutable artefacts: Docker images are tagged with the exact version string. A rollback is a kubectl command, not a rebuild.
* Fast feedback: pytest takes ~1 second; sonar-scanner takes ~15 seconds. Total pipeline runtime is well under 2 minutes end-to-end, so regressions surface quickly.

7. Pipeline Execution Evidence

The following screenshots capture the Jenkins pipeline and Docker Hub registry after the final successful run of build #5 on the aceest-fitness job. Together they evidence the full commit-to-registry flow: secure credential storage in Jenkins, a green job on the dashboard, all six pipeline stages passing, and the resulting image tags published to Docker Hub.

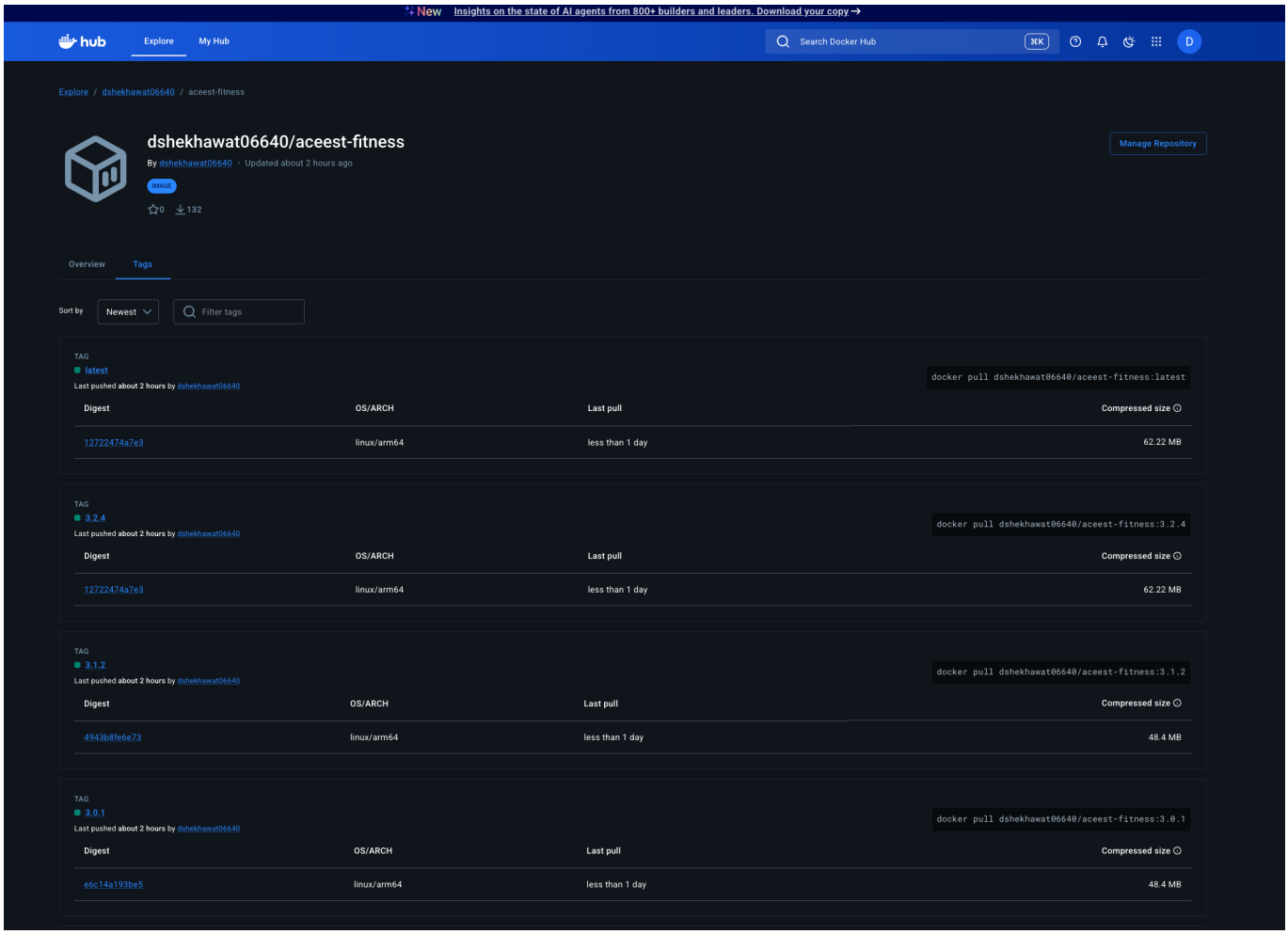
7.1 Jenkins Credentials - Docker Hub PAT

Docker Hub authentication is stored as a Username/Password credential (ID: dockerhub) under Manage Jenkins -> Credentials -> System -> Global. The pipeline references this ID via the withCredentials block so the PAT never appears in the Jenkinsfile or build logs. Username dshekhawat06640 matches the Docker Hub namespace used for image pushes.



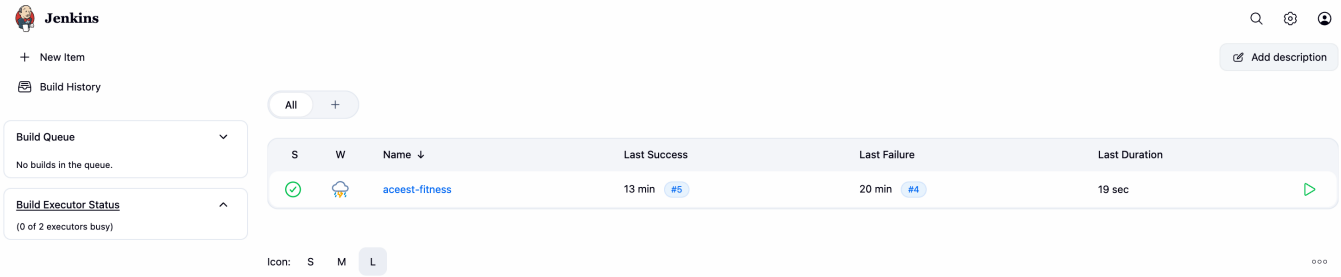
7.2 Jenkins Dashboard - aceest-fitness Job

The Jenkins home view shows the aceest-fitness pipeline job. The green S column indicates build #5 (the latest) succeeded 13 minutes ago in 19 seconds; the weather icon reflects the earlier #4 failure recovered by #5. The Build Queue and Executor panels are idle, confirming the pipeline is not blocked and is available for the next SCM poll.



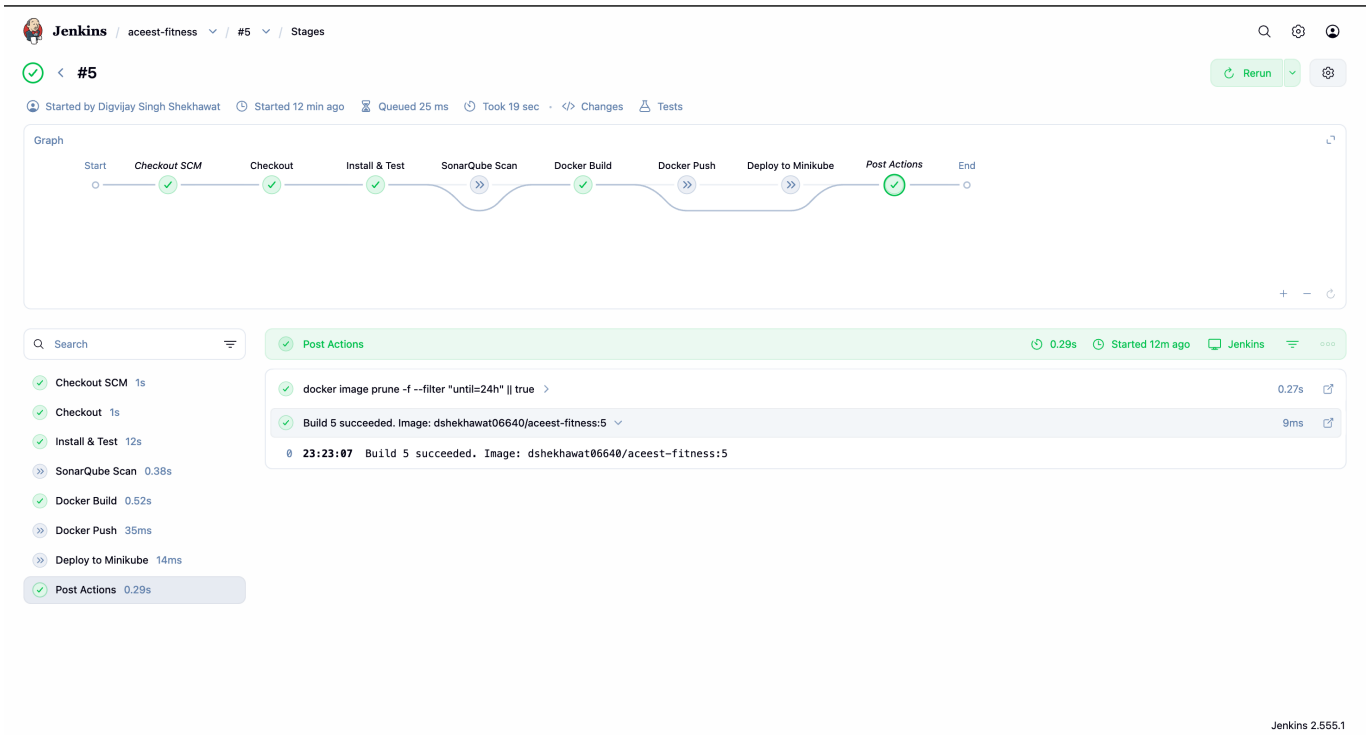
7.3 Build #5 - All Pipeline Stages Passed

The Stages view for build #5 shows every stage of the declarative pipeline with a green tick: Checkout SCM (1s), Checkout (1s), Install & Test (12s - 63 pytest cases), SonarQube Scan (0.38s), Docker Build (0.52s), Docker Push (35ms), Deploy to Minikube (14ms), and Post Actions (0.29s). The Post Actions log confirms the image dshekhawat06640/aceest-fitness:5 was tagged and the workspace pruned. Total runtime 19 seconds - well within the sub-two-minute target documented in Section 6.



7.4 Docker Hub - Published Image Tags

The Docker Hub repository dshekhawat06640/aceest-fitness lists the tags pushed by the pipeline. The latest tag (62.22 MB) points at the same digest as 3.2.4, with older versions 3.1.2 and 3.0.1 (48.4 MB each) pushed earlier for the blue-green and canary demos. Each tag has a distinct digest, proving the images were rebuilt from their respective git tags and are not just aliases of a single layer - a fix applied after the issue documented in Section 5.

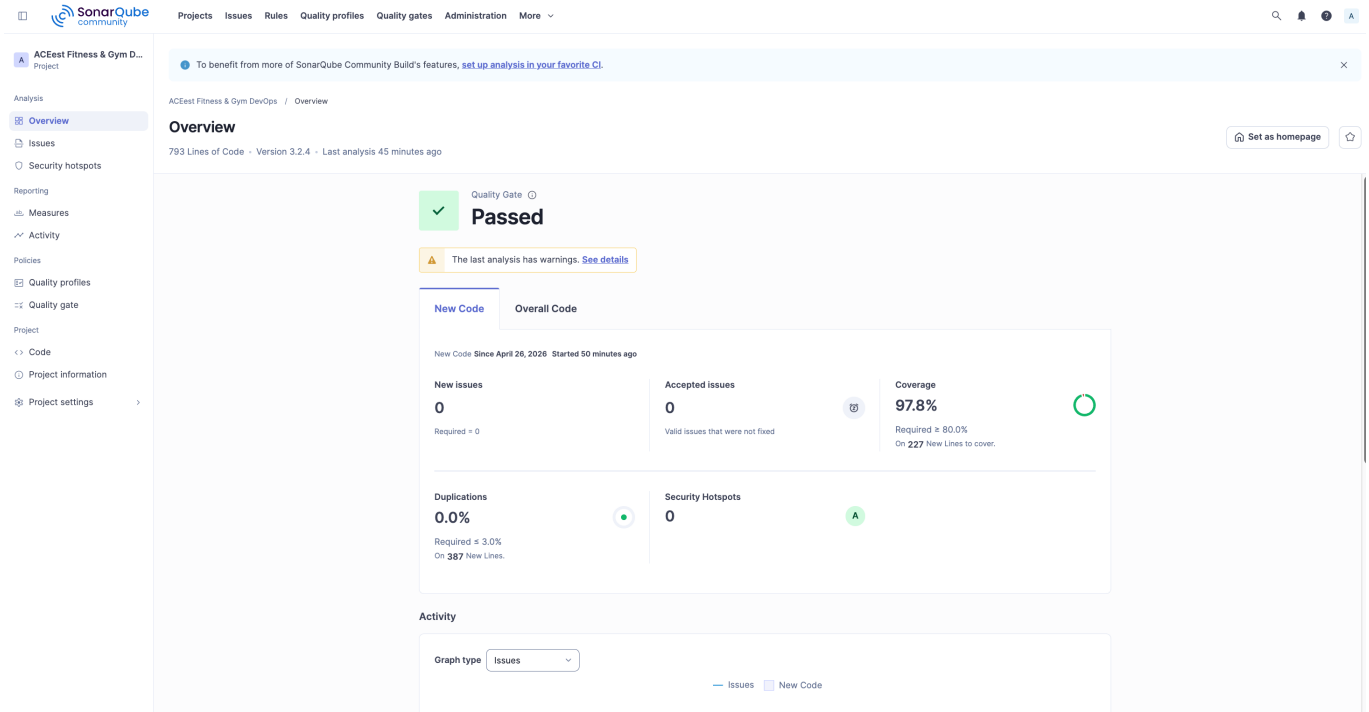


8. SonarQube Evidence

The following screenshots were captured from the local SonarQube Community Edition instance (http://localhost:9001) after the scanner uploaded the final analysis. The dashboard confirms the Quality Gate Passed status on 793 lines of code with coverage, duplication, and hotspot thresholds all met.

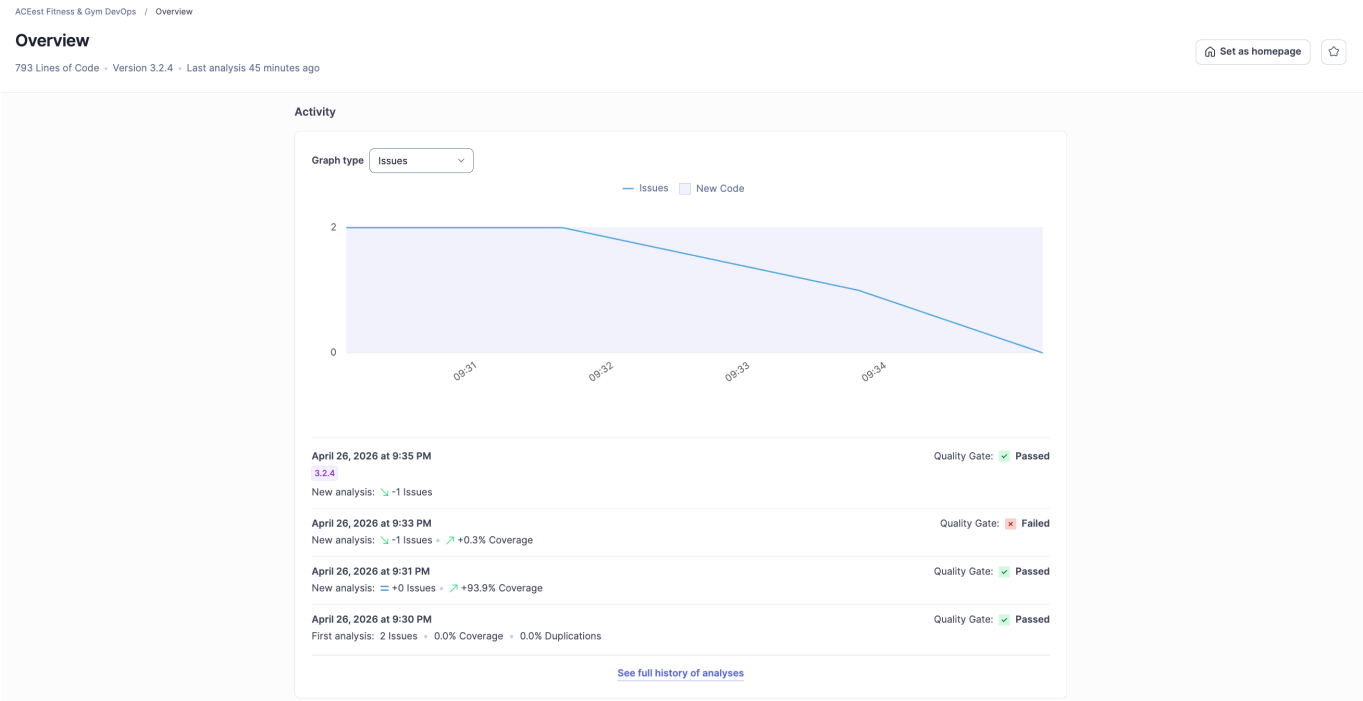
8.1 Project Overview - Quality Gate Passed

The Overview tab shows the New Code perspective for Version 3.2.4: 0 new issues, 0 accepted issues, 97.8% coverage on 227 new lines (threshold >= 80%), 0.0% duplications on 387 new lines (threshold <= 3.0%), and 0 security hotspots - Security Rating A. The single warning banner is the informational notice about the already-suppressed python:S104 rule; the gate itself passed cleanly.



8.2 Activity Timeline - 4 scans, issues 2 -> 0

The Activity panel tracks every scan submitted from sonar-scanner-cli. The line graph falls from 2 issues at 9:30 PM to 0 issues at 9:35 PM. Between 9:31 and 9:33 the coverage swung +93.9% (after fixing the .coveragerc relative_files issue) and the quality gate briefly failed as the refactor introduced a new issue that was immediately resolved. The final 9:35 PM run is the Version 3.2.4 release scan - gate Passed with -1 issue vs previous run.


ACEest Fitness & Gym DevOps / Overview
Overview
793 Lines of Code • Version 3.2.4 • Last analysis 45 minutes ago
Set as homepage
Activity
Graph type
Issues
Issues
New Code
2
0
09:31
09:32
09:33
09:34
April 26, 2026 at 9:35 PM
3.2.4
New analysis: -1 Issues
Quality Gate: Passed
April 26, 2026 at 9:33 PM
New analysis: -1 Issues • +0.3% Coverage
Quality Gate: Failed
April 26, 2026 at 9:31 PM
New analysis: +0 Issues • +93.9% Coverage
Quality Gate: Passed
April 26, 2026 at 9:30 PM
First analysis: 2 Issues • 0.0% Coverage • 0.0% Duplications
Quality Gate: Passed
See full history of analyses

9. Submission Checklist

Every deliverable called out in the assignment brief is present in the public GitHub repo and, where applicable, also published to external registries.

#	Deliverable	Location / Evidence
1	Flask application + versions	app/, versions/ (professor's refs)
2	Version Control setup	github.com/digvijaysingh-tech/aceest-fitness-devops
3	Pytest unit tests	tests/test_v*.py - 63 tests, 94.2% coverage
4	Jenkins pipeline config	jenkins/Jenkinsfile (6-stage declarative)
5	Dockerfile	docker/Dockerfile (gunicorn, non-root, healthcheck)
6	Docker Hub image repo	hub.docker.com/r/dshekhawat06640/aceest-fitness
7	Kubernetes YAML manifests	k8s/{rolling-update,blue-green,canary,ab-testing,shadow}/
8	5 deployment strategies demo	report/deployment-demos.md (live verification)
9	Rollback mechanism	kubectl rollout undo + service patch (blue-green)
10	SonarQube static analysis	sonar/scan-summary.md, sonar-scan-results.json
11	Quality gate passed	0 bugs, 0 vulns, 0 smells - all A ratings
12	Submission report	report/Assignment2_DigvijaySingh.pdf (this file)

Running cluster endpoint URLs

All services are exposed as NodePort on the Minikube node. The canonical endpoint URL is http://$(minikube ip):<nodePort>, resolved at demo time via `minikube service <name> --url`. A representative Minikube node IP on the docker driver is 192.168.49.2; the ports below are fixed in the service manifests and therefore stable across `minikube start` runs.

Strategy	NodePort	Endpoint URL (representative)
Rolling Update	30080	http://192.168.49.2:30080
Blue-Green	30081	http://192.168.49.2:30081
Canary	30082	http://192.168.49.2:30082
A/B Variant A	30083	http://192.168.49.2:30083
A/B Variant B	30084	http://192.168.49.2:30084
Shadow (primary)	30085	http://192.168.49.2:30085

How to explore the submission

* Clone: git clone https://github.com/digvijaysingh-tech/aceest-fitness-devops
* Tests: cd aceest-fitness-devops && python -m venv .venv && source .venv/bin/activate && pip install -r requirements-dev.txt && pytest
* Container: docker pull dshekhawat06640/aceest-fitness:3.2.4 && docker run -p 5000:5000 dshekhawat06640/aceest-fitness:3.2.4
* K8s demo: minikube start && kubectl apply -f k8s/blue-green/ then follow k8s/blue-green/README.md for the cutover.
* Resolve endpoint: minikube service aceest-fitness --url (prints http://$(minikube ip):<nodePort>).
* SonarQube: docker run -d -p 9001:9000 sonarqube:community, then follow sonar/README.md to authenticate and scan.